COMPTIA PENTEST+ · PT0-003 · STUDY GUIDE

Domain 4 — The *Why* Behind the Attacks

Ten root-cause principles that explain the whole domain. Understand the mechanism once, and the individual attacks — and their fixes — become things you can *reason to*, not memorize.

Learn the why, not just the what

Why a "why" guide? CompTIA's questions are usually *one detail off* — a wrong flag, the wrong layer, a plausible-but-weaker fix. If you memorized a fact, all four options look familiar and you guess. If you understand the *mechanism*, the wrong answers *contradict the mechanism* and eliminate themselves.

Each principle below states a memorable **rule**, explains the **WHY**, shows the exam attacks that are the *same idea in disguise*, explains **WHY THE FIX WORKS** (not just what it is), and gives the **DISTRACTOR TELL** the mechanism hands you.

1 Code and data get confused

"If attacker input can reach an interpreter, it becomes instructions."

THE WHY: Every program handles two kinds of bytes — *data* (what the user typed) and *instructions* (what the program should do). Injection is one bug: attacker data crosses into the instruction channel and gets *interpreted* instead of treated as inert text. The entire injection family is the same flaw aimed at different interpreters.

Exam attack	...is the same bug at this interpreter
SQL injection	The SQL query engine.
Command injection	The OS shell.
XSS	The browser's HTML/JS parser.
SSTI	The server-side template engine.
LDAP injection	The directory query parser.
Path traversal / LFI	The filesystem path resolver.

WHY THE FIX WORKS: Every fix *restores the data/instruction separation*. Parameterized queries bind input as a *value* that can never become SQL syntax. Output encoding tells the browser "this is text, not markup." Safe APIs avoid the shell entirely. The fix isn't "clean the input" — it's "make the input structurally incapable of being code."

Distractor tell: an option that merely *filters/validates* input is always weaker than one that *separates code from data*. That's why **parameterized queries** — not "input validation" or "a WAF" — is the *best* SQLi answer.

2 The confused deputy

"Don't attack the target — trick something the target already trusts into doing it for you."

THE WHY: A "deputy" is a component with more access than you have. A confused-deputy attack makes that trusted component use *its* access on *your* behalf. Three separate-looking exam attacks are all this idea:

Exam attack	The deputy you confuse
SSRF	The *server* — you make it fetch internal URLs it can reach but you can't (e.g., cloud metadata).
CSRF	The *victim's browser* — it auto-attaches their cookies, so your forged request rides their session.
NTLM relay	The *authentication exchange* — you forward the victim's auth to a host that trusts it.

WHY THE FIX WORKS: Each fix forces the deputy to *verify intent or identity* before acting. SSRF: allowlist what the server may request. CSRF: an anti-CSRF token proves the request came from your own page — something a foreign page can't read. Relay: SMB/LDAP *signing* binds the auth to its session so it can't be forwarded.

> **Distractor tell:** the winning fix adds *proof the action was intended by the right party*. Options that only log, monitor, or encrypt the payload don't stop a confused deputy.

3 Never trust the client

"Anything the client can touch, the attacker can forge."

THE WHY: URL parameters, hidden fields, cookies, tokens, and even the order of requests are all attacker-controllable. Access control breaks whenever the server *assumes* the client won't tamper. Broken access control is consistently the highest-impact web category for exactly this reason.

- **IDOR** — server trusts the object ID in the request and never re-checks ownership.
- **JWT manipulation** — server trusts a token it didn't properly verify (`alg=none`, weak key).
- **Mass assignment / hidden-field tampering** — server trusts fields the client shouldn't be allowed to set.
- **Forced browsing** — server trusts that users won't visit un-linked URLs.

WHY THE FIX WORKS: Authorization must be re-decided *server-side, per request, per object*, using something the client can't forge — the authenticated session's identity — and tokens must be cryptographically verified. The mechanism moves the decision out of the attacker's reach.

> **Distractor tell:** "client-side validation," "hide the endpoint," or "obfuscate the ID" are *always wrong* for access control. If the defense lives where the attacker sits, it isn't a defense.

4 Possession of the secret *is* identity

"In Windows/AD, you don't need the password — you need the material that proves you know it."

THE WHY: Windows and Kerberos were designed so that proving you hold the *secret material* — a password hash, a ticket, a token — proves you're the user, without ever needing the plaintext. That enables single sign-on, but it means *stealing the material = becoming the user*, no cracking required. This is why the "pass-the-*" family exists at all.

Exam attack	The bearer secret you reuse
Pass-the-hash	The NTLM hash itself answers the challenge — it *is* the credential.
Pass-the-ticket	A Kerberos ticket is a bearer proof; hold it, use it.
Kerberoasting	A service ticket is encrypted with the service account's key — possession lets you crack it *offline* at leisure.
Pass-the-token	Session/OAuth tokens are bearer tokens — steal it, you're in.

WHY THE FIX WORKS: The fixes break the "possession = identity" equation. MFA adds a factor that isn't sitting on the host and can't be replayed remotely. Credential Guard / LSASS protection stops the theft. Short-lived, context-bound tokens shrink the reuse window. Long service-account passwords make offline cracking infeasible.

> **Distractor tell:** "change the user's password" does *not* instantly stop pass-the-hash, because the *hash* is the currency — understanding that the material (not the plaintext) authenticates is the whole point.

---

5 Guessable secret + unlimited attempts

"Passwords fail on two axes: they're predictable, and nothing stops you trying."

THE WHY: Two independent weaknesses power every password attack — *predictability* (humans pick common passwords and reuse them) and *attempt economics* (there's often no cost to another guess). Each attack leans on one axis:

- **Brute-force / dictionary** — attack predictability with raw volume.
- **Password spraying** — flips the tactic: one common password across many users, deliberately staying *under* the lockout threshold. Its whole reason for existing is *lockout evasion*.
- **Credential stuffing** — reuse across sites means one breach unlocks many accounts.
- **Offline cracking** — once you hold the hashes, attempt economics collapse: millions/sec, and *no lockout applies*.

WHY THE FIX WORKS: MFA defeats "the password alone is enough." Lockout and rate-limiting restore attempt cost *online*. Strong *slow* hashing (bcrypt/argon2) restores attempt cost *offline*. Banning breached/common passwords attacks the predictability axis directly.

> **Distractor tell:** account lockout does *nothing* against offline cracking — there's no server to lock. So the fix for a stolen hash database is **strong/slow hashing**, never "lockout policy."

6 First answer wins, and nobody checks ID

"LAN protocols were built to be helpful on a trusted network — so they believe whoever answers."

THE WHY: ARP, LLMNR/NBT-NS/mDNS, DHCP, and classic DNS all optimize for helpfulness and *trust the first/any responder* with no authentication. On a network you share, that means whoever answers fastest — or sits in the path — is believed. This single design assumption powers the whole on-path/poisoning/relay chain.

- **LLMNR/NBT-NS poisoning** — you answer name-resolution broadcasts; victims hand you their NetNTLM auth.
- **ARP spoofing** — you claim to be the gateway; traffic routes through you.
- **Rogue DHCP** — you hand out your own IP as the gateway/DNS.

WHY THE FIX WORKS: Every fix adds the missing *verification of the responder*. Disabling LLMNR removes the trusting protocol. SMB signing means captured auth can't be relayed. DHCP snooping and Dynamic ARP Inspection let the switch verify who's *allowed* to answer. DNSSEC signs the records.

> **Distractor tell:** the fix that *verifies who is allowed to respond* beats one that just encrypts traffic — encryption doesn't help once you've willingly connected to the attacker.

---

7 Convenient by default is insecure by default

"Products ship to work out of the box — which means they ship trusting their environment."

THE WHY: Ease-of-setup trades away security: permissive defaults, well-known credentials, and services that trust anything local. The attack is simply *using the convenience as intended*.

- **Default credentials / configs** — known to everyone, including you.
- **WPS** — a usability shortcut that bypasses the strong PSK.
- **Cloud metadata service (IMDS)** — trusts *any* local process, so an SSRF or RCE inherits the instance's cloud role and steals its credentials.
- **Public storage buckets** — default sharing plus human error.

WHY THE FIX WORKS: The fixes replace "trust by default" with "least privilege by intent": change defaults, disable convenience features, block public access. `IMDSv2` specifically requires a signed session token, so a *blind* SSRF can't just GET the credentials — the mechanism adds a step the attacker can't perform.

> **Distractor tell:** beware answers that reason "it's internal, so it's safe." The metadata service and default creds are exactly where that assumption gets punished.

---

8 Isolation is only as strong as its weakest boundary

"Escalation happens wherever a low-privilege actor can influence a high-privilege action."

THE WHY: Systems separate low-trust from high-trust with boundaries — user vs SYSTEM/root, container vs host, restricted shell vs full shell. Privilege escalation and escape happen at any point where that boundary has a hole the attacker can *reach and control*. The specific technique is just *which hole*.

- **Unquoted service path, weak service ACL, SUID binary, over-broad capability, writable cron** — each is a spot where a low-priv user influences a high-priv execution.
- **Container escape** — the isolation (namespaces/capabilities) is incomplete or over-permissioned (a privileged container).
- **Shell / kiosk escape** — the "jail" left behind a binary that spawns a real shell (GTFOBins).

WHY THE FIX WORKS: Every fix *removes the low-priv actor's influence over the boundary*: quote the path, tighten the ACL, drop SUID/capabilities, run containers unprivileged. Same mechanism, restated per case.

> **Distractor tell:** the vulnerable element is always something a *low-priv user can write to or control* that a *high-priv context later trusts*. Find that in the scenario and the answer follows.

---

9 Humans run on shortcuts

"Social engineering doesn't exploit code — it exploits reliable mental shortcuts."

THE WHY: People lean on cognitive shortcuts — authority, urgency, scarcity, reciprocity, social proof, familiarity. The attacker manufactures a situation where the shortcut fires *before* deliberate thought does. Phishing, vishing, and smishing are just delivery channels for the same psychology; spear-phishing and whaling add OSINT-driven tailoring to make the trigger more reliable; **MFA fatigue** weaponizes *habituation* — spam prompts until "approve" becomes reflex.

WHY THE FIX WORKS: You can't patch a human, so you patch the *process*: training plus procedure inserts a deliberate verification step that interrupts the shortcut — callback procedures, out-of-band approval for transfers, number-matching MFA that defeats reflexive approval.

> **Distractor tell:** the correct control usually adds *independent verification*, not just "tell users to be more careful." A process that forces a second, deliberate check beats an awareness-only answer.

---

10 When the math stops holding

"Crypto attacks appear when one of its assumptions quietly breaks."

THE WHY: Cryptographic security rests on assumptions — collisions are infeasible, keys are unpredictable, keystreams aren't reused. An attack is the discovery that one assumption no longer holds for a given algorithm.

- **Collision attack** — a weak hash (MD5/SHA-1) lets two inputs share a digest, defeating integrity and signatures.
- **WEP cracking** — IV reuse leaks the keystream.
- **Fast / unsalted hashes** — make offline cracking trivial (ties back to Principle 5).

WHY THE FIX WORKS: Adopt algorithms whose assumptions still hold — SHA-256+, WPA3, salted slow KDFs. The fix is literally "restore the broken assumption."

> **Distractor tell:** "add more rounds of the broken algorithm" or "keep the cipher but lengthen the key" is never the answer — you *replace* the algorithm.

How to study with the "why"

- **Read the scenario for the mechanism, not the keyword.** Ask "which trust is being abused here?" — an interpreter, a deputy, the client, a bearer secret, a responder, a default, a boundary, a human, or a broken assumption. That single question routes you to the right principle.
- **Make the fix explain itself.** For every remediation, say *why* it closes the mechanism. If you can't, you've memorized it, not learned it.
- **Use the distractor tells actively.** On each question, try to eliminate two options because they *contradict the mechanism* before you pick.
- **Pair this with your other two guides.** Surface = where attacks live; objective map = the blueprint; this = why they work. Three angles on the same 35%.

Authorized use only. These principles are for penetration testing within a defined scope and signed rules of engagement. Testing systems you don't own or aren't authorized to assess is illegal. Understanding *why* an attack works is also what lets you write the remediation that actually closes it in your report.

CompTIA PenTest+ PT0-003 · Domain 4 (Attacks & Exploits, 35%) · Organized by root-cause principle: the mechanism → the attacks that share it → why the fix works → the distractor tell. A companion to the attack-surface and objective-map guides.